

Laurea Magistrale in Informatica

Curriculum SICUREZZA INFORMATICA (opzionale)

Curriculum CLOUD COMPUTING (opzionale)

**Curriculum Software Engineering and IT Management
(obbligatorio – I ANNO)**

Corso di

Compilatori

6 CFU (48 ore) lezioni

3 CFU (24 ore) laboratorio/esercitazioni

Prof. Gennaro Costagliola

Cosa è un Compilatore?

Un compilatore è un **programma informatico** che traduce

- una serie di istruzioni scritte in un determinato **linguaggio di programmazione** (*codice sorgente*)
- in istruzioni di un **altro linguaggio** (*codice oggetto*), di solito interpretabile ed eseguibile da un computer.

Una mappa dei L.P.

Dal 1957 sono stati definiti ed implementati più di 2300 linguaggi di programmazione di cui molti sono estinti o in via di estinzione

Mother Tongues

Tracing the roots of computer languages through the ages

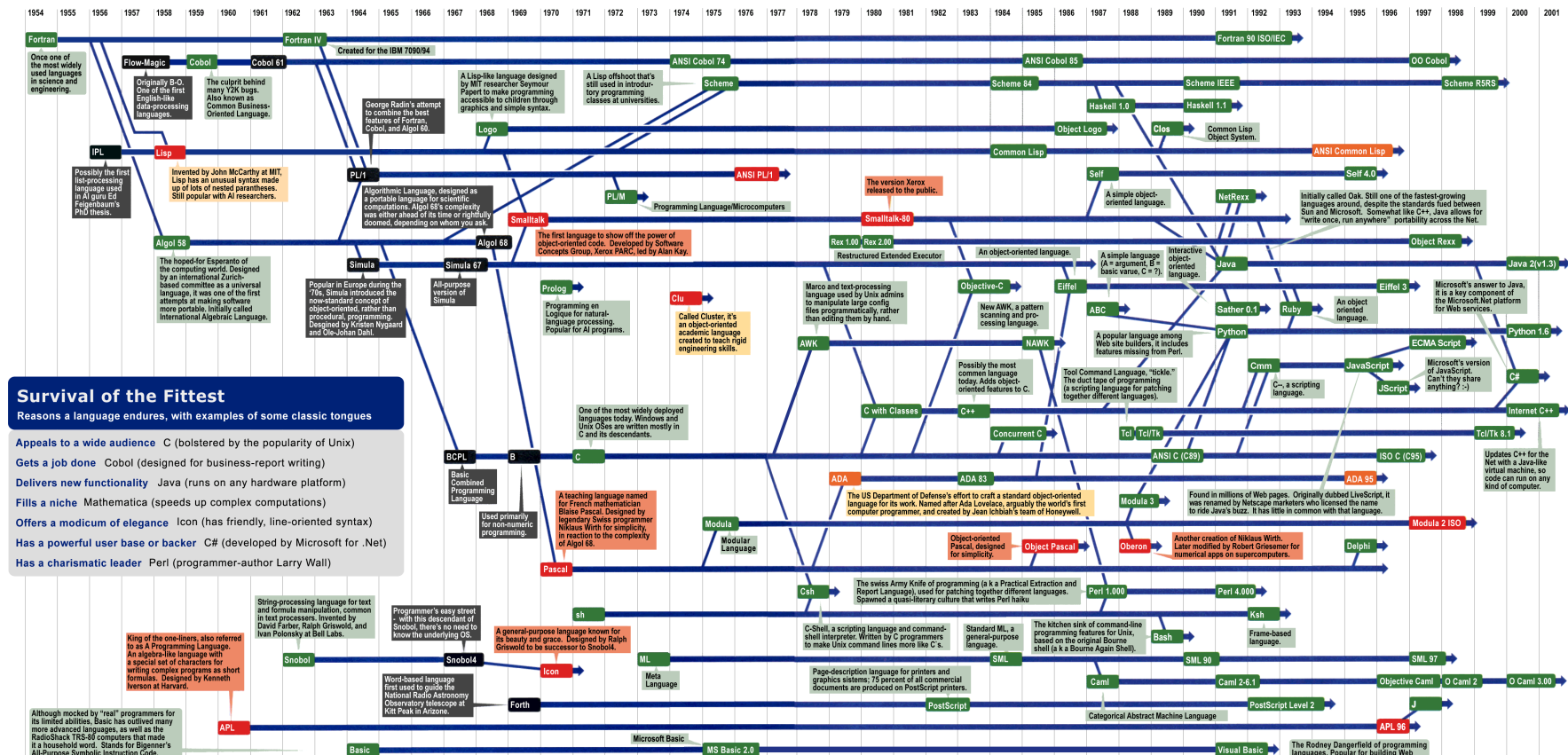
Just like half of the world's spoken tongues, most of the 2,300-plus computer programming languages are either endangered or extinct. As powerhouses C/C++, Visual Basic, Cobol, Java and other modern source codes dominate our systems, hundreds of older languages are running out of life.

An ad hoc collection of engineers-electronic lexicographers, if you will-aim to save, or at least document the lingo of classic software. They're combing the globe's 9 million developers in search of coders still fluent in these nearly forgotten lingua frangas. Among the most endangered are Ada, APL, B (the predecessor of C), Lsp, Oberon, Smalltalk, and Simula.

Code-raker Grady Booch, Rational Software's chief scientist, is working with the Computer History Museum in Silicon Valley to record and, in some cases, maintain languages by writing new compilers so our ever-changing hardware can grok the code. Why bother? "They tell us about the state of software practice, the minds of their inventors, and the technical, social, and economic forces that shaped history at the time," Booch explains. "They'll provide the raw material for software archaeologists, historians, and developers to learn what worked, what was brilliant, and what was an utter failure." Here's a peek at the strongest branches of programming's family tree. For a nearly exhaustive rundown, check out the Language List at [HTTP://www.informatik.uni-freiburg.de/java/misc/lang_list.html](http://www.informatik.uni-freiburg.de/java/misc/lang_list.html). - Michael Mendeno

Key

- 1954 Year Introduced
- Active: thousands of users
- Protected: taught at universities; compilers available
- Endangered: usage dropping off
- Extinct: no known active users or up-to-date compilers
- Lineage continues

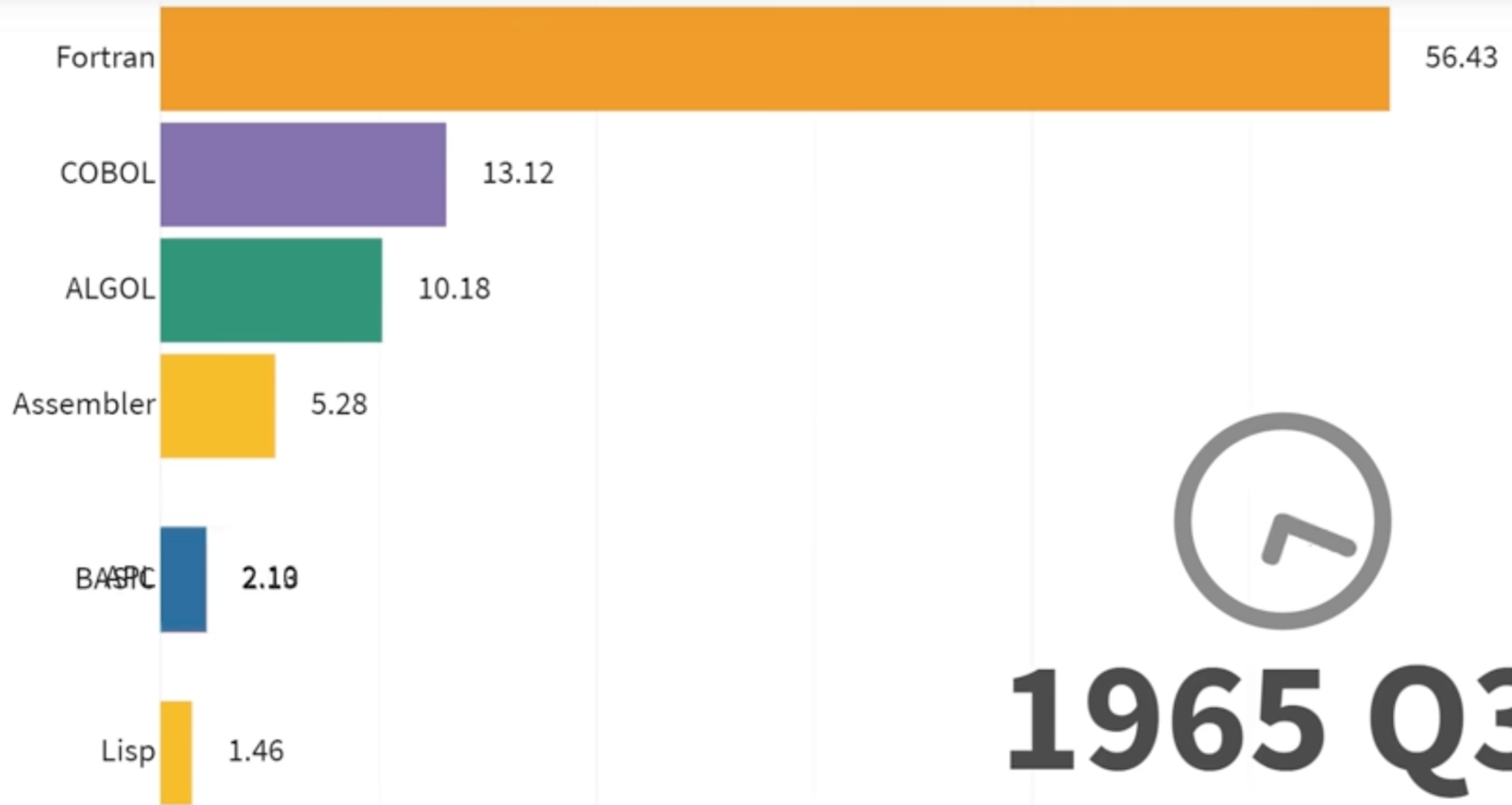




Most Popular Programming Languages 1965 - 2019

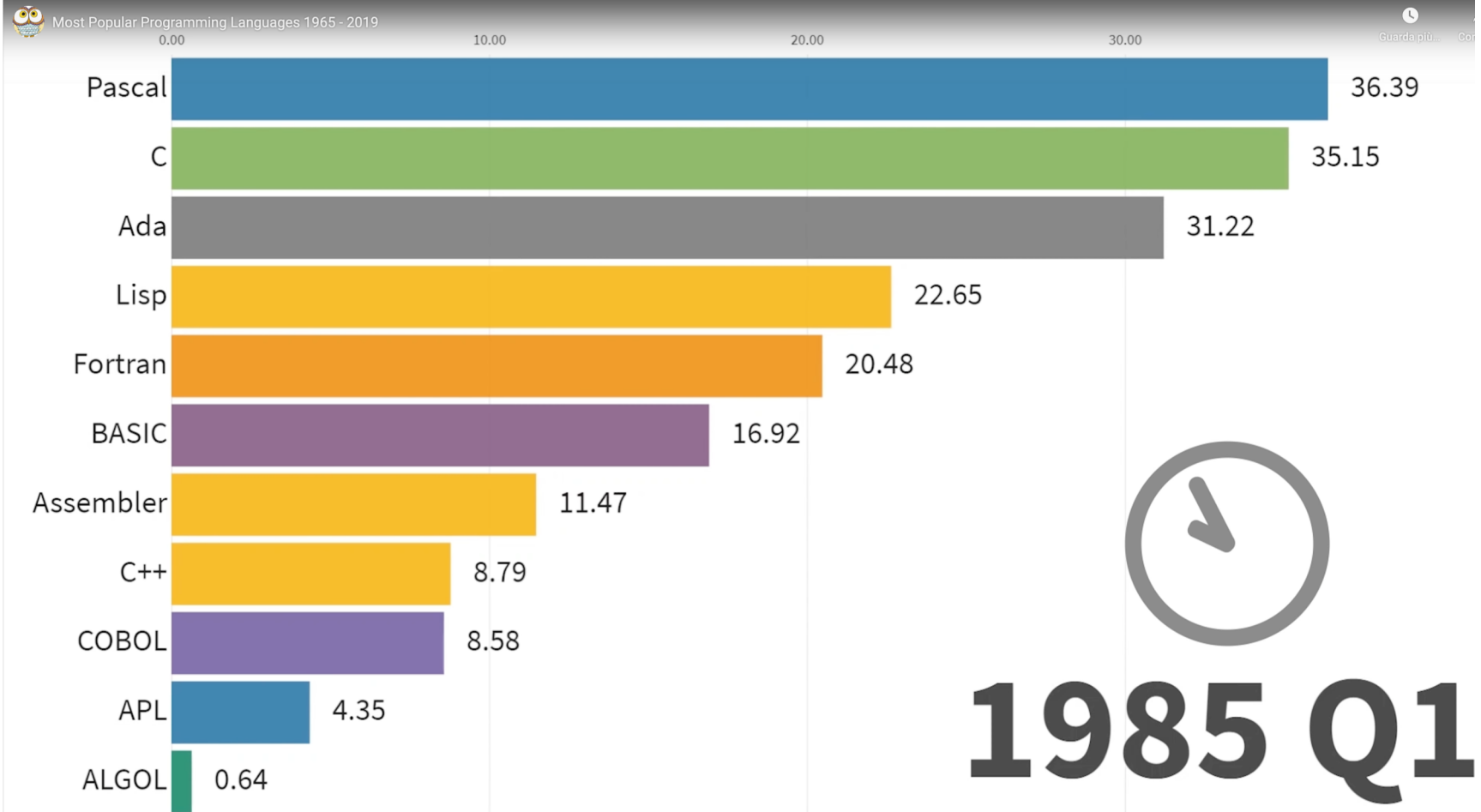


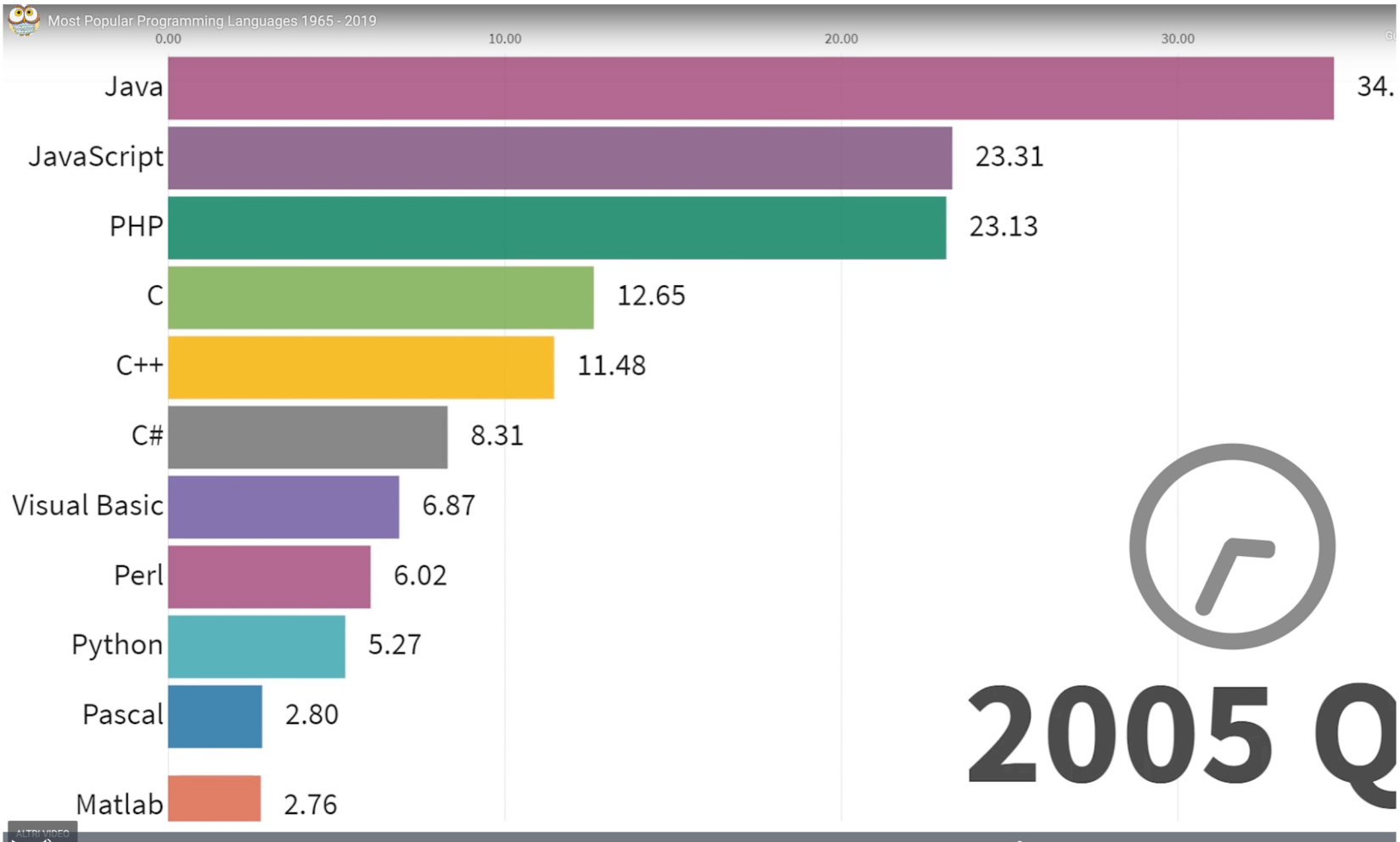
Guarda più... Condividi



1965 Q3

Wondershare PDFElement

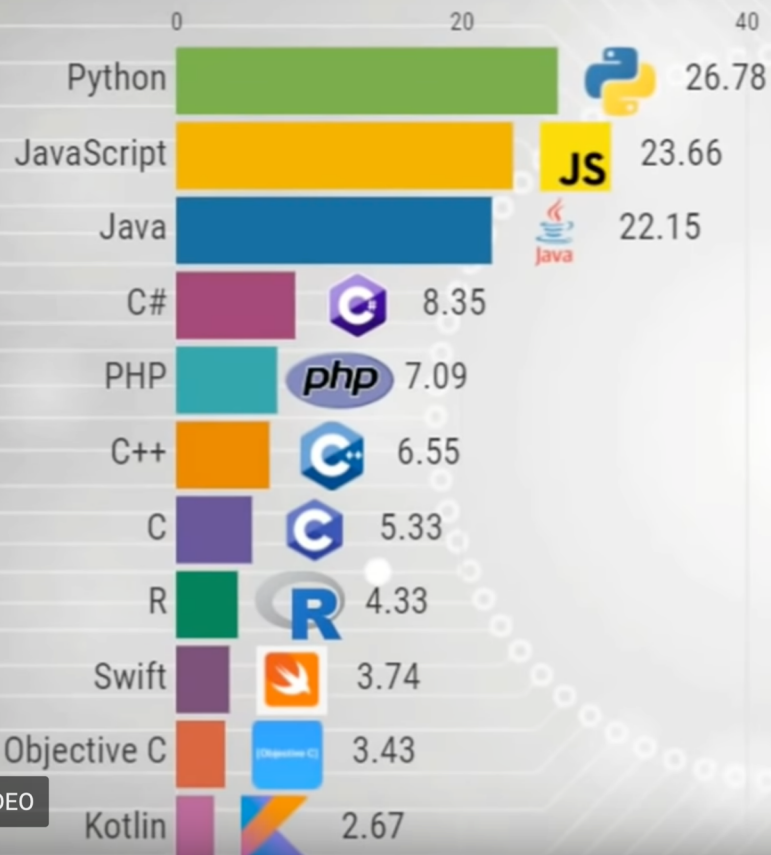




The Most Popular Programming Languages - 1965/2020

Most Popular Programming Languages

Guarda più tardi Condividi Informaz



2020 Q2

Perchè studiare i Compilatori

Comprendere la progettazione e lo sviluppo di un linguaggio di programmazione

E di conseguenza:

- Migliorare l'uso dei linguaggi di programmazione utilizzati.
- Facilitare l'apprendimento di un nuovo linguaggio di programmazione
- Utilizzare in un unico paradigma concetti di **strutture dati, algoritmi, architetture hardware ed assembly, programmazione di alto livello, teoria della computazione e sistemi operativi.**

(T. Pratt – M.V. Zelkowitz, Program Languages Design and implementation)

ACM Computer Science Curriculum

“Several industrialists passed very positive comment about compiler courses. Although many companies do not engage in anything related to compilers, **compiler writing tended to be seen as a microcosm for realistic software development.** So good compiler writers are often seen as desirable; they tend to be good software engineers.”

Struttura logica di un compilatore

Cosa copriremo nel corso:

- Dall'analisi lessicale alla generazione del codice intermedio e cenni di generazione del codice macchina ed ottimizzazione
- Sviluppo delle prime tre fasi di un compilatore (lessicale, sintattico e semantico) di un semplice linguaggio di programmazione

position = initial + rate * 60

Lexical Analyzer

$\langle id, 1 \rangle \langle = \rangle \langle id, 2 \rangle \langle + \rangle \langle id, 3 \rangle \langle * \rangle \langle 60 \rangle$

Syntax Analyzer

$\langle id, 1 \rangle = \langle id, 2 \rangle + \langle id, 3 \rangle * 60$

Semantic Analyzer

$\langle id, 1 \rangle = \langle id, 2 \rangle + \langle id, 3 \rangle * \text{inttofloat}(60)$

Intermediate Code Generator

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Code Optimizer

```
t1 = id3 * 60.0
id1 = id2 + t1
```

Code Generator

```
LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```

1	position	...
2	initial	...
3	rate	...

SYMBOL TABLE

Figure 1.7: Translation of an assignment statement

Materiale didattico

- V. Aho, S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, Addison-Wesley, Second Edition.
- Materiale online pubblicato sul sito del corso

